



The `get_max()` function is a simple PL/PgSQL function that is installed in the Docker container of this chapter and is used only to demonstrate how PostgreSQL is smart enough to remove all the dependent objects when dropping an extension.

To summarize, you have learned how to manually manage an extension, from installing it to upgrading it or removing it; in the next section, you will learn how to perform the same steps in a more automated way.

Exploring the PGXN client

The PGXN client is an external application, written in Python, that works as a command-line interface for PGXN. The application, named `pgxnclient`, works by means of commands, which are actions such as `install`, `download`, `uninstall`, and so on, allowing a database administrator to interact with PGXN and extensions.



To some extent, `pgxnclient` works the same as the `cpan` (or `cpanm`) command for Perl, `zef` for Raku, `pip` for Python, and so on.

Being an external application means that `pgxnclient` is not distributed with PostgreSQL, and therefore you need to install it on your machine before you can use it. Installing `pgxnclient` is not mandatory in order to use PostgreSQL extensions, but it can make your life a lot easier.

In the following subsections, you will see how to install `pgxnclient` on main Unix and Unix-like operating systems, but before that, it is important to let you know that, once it is installed, you will find two executables on your system: `pgxn` and `pgxnclient`. You can think of those executables as aliases of one another, even if this is not really true (one wraps the other); however, you can use either one you please, obtaining the very same result. In this chapter, we will use `pgxn` as the main executable.



In the Docker images in the book's repository, the `pgxnclient` program is already installed, so you don't need to install it.

Detailed instructions on how to install `pgxnclient` are beyond the scope of this book, therefore if you are not able to get the application working properly, please double-check the project documentation and installation instructions.

Installing pgxnclient on Debian GNU/Linux and derivatives

pgxnclient is packaged for Debian GNU/Linux and derivatives, and that means you can simply ask apt to install it:

```
$ sudo apt install pgxnclient
```

Once the program has been installed, you can simply test it with the `--version` option, which will print the version number you installed:

```
$ pgxn --version  
pgxnclient 1.3.2
```

Installing pgxnclient on Fedora Linux and Red Hat-based distributions

pgxnclient is packaged for Fedora as well, so you can install it with the operating system package manager:

```
$ sudo dnf install -y pgxnclient
```

Once the process is complete, you can query the application to verify it is actually working:

```
$ pgxn --version  
pgxnclient 1.3.2
```

Installing pgxnclient on FreeBSD

pgxnclient is packaged for FreeBSD, so you can install it via the pkg tool or via the software ports. The fastest way is by using pkg, and all you have to do is ask to install the program:

```
$ sudo pkg install --yes pgxnclient
```

Installing pgxnclient from sources

You can always install pgxnclient from sources, even if this is suggested only if you are on an operating system that does not provide a packaged version, or if the version is out of date with regard to your needs. You can download a compressed version of the latest release from the official project GitHub repository, for example:

```
$ wget https://github.com/pgxn/pgxnclient/archive/refs/tags/v1.3.2.zip
```

Once you have the compressed archive, you need to decompress it and enter the directory that will be created with it – named after the version of PGXN you have downloaded – in our case, `pgxnclient-3.1.2`. Once you are in the directory, executing the `setup.py` Python script will allow you to install the application:

```
$ unzip v1.3.2.zip
$ cd pgxnclient-1.3.2
$ sudo python setup.py install
...
Finished processing dependencies for pgxnclient==1.3.2
```

Once you have completed the installation, you can query the application to verify that it is working:

```
$ pgxn --version
pgxnclient 1.3.2
```

The `pgxnclient` command-line interface

The PGXN client application provides a command-line interface similar to other command-based applications, such as `cpanm` and `git`. You can get a list of the main commands by asking for help:

```
$ pgxn help
usage: pgxn [--version] [--help] COMMAND ...

Interact with the PostgreSQL Extension Network (PGXN).

optional arguments:
  --version  print the version number and exit
  --help     show this help message and exit

available commands:
  COMMAND    the command to execute. The complete list is available using
              'pgxn help --all'. Builtin commands are:
  check      run a distribution's test
  download   download a distribution from the network
  help       display help and other program information
  info       print information about a distribution
  install    download, build and install a distribution
```

```
load      load a distribution's extensions into a database
mirror    return information about the available mirrors
search    search in the available extensions
uninstall

          remove a distribution from the system
unload    unload a distribution's extensions from a database
```

Usually, you will use the following subset of commands:

- `search` to search for distributions using keywords
- `info` to have a closer look at an extension
- `download` to download (but not install) an extension
- `install` to download and install an extension in the cluster
- `load` to execute `CREATE EXTENSION` against a specific database
- `unload` to execute `DROP EXTENSION` against a specific database
- `uninstall` to remove an extension from a cluster

The smallest set of commands you will probably use are `search`, `install`, and `uninstall`.

For every command, you can get more detailed help if you specify the command as an argument to the `help` command. For example, to get more information about the `search` command, you can do the following:

```
$ pgxn help search
usage: pgxn search [--help] [--mirror URL] [--verbose] [--yes]
                  [--docs | --dist | --ext]
                  TERM [TERM ...]

search in the available extensions

positional arguments:
  TERM                a string to search

optional arguments:
  --help              show this help message and exit
  --docs              search in documentation [default]
  --dist              search in distributions
  --ext               search in extensions
```

```
global options:
  --mirror URL  the mirror to interact with [default: https://api.pgxn.
org/]
  --verbose      print more information
  --yes          assume affirmative answer to all questions
```

In the following sections, you will see how to use PXGN effectively to install an extension.

Installing extensions

Usually, the workflow for getting an extension up and running involves a few steps. First, you need to find out which extension to use, which version, and the compatibility with your cluster. Once you have found out the extension you need, you have to install it in the cluster.

Installing it in the cluster really means *deploying* it in the PostgreSQL directories, that is, moving all the extension-related files and libraries into the shared directory of the cluster so that PostgreSQL can seek the code required to run the extension.

Lastly, you need to create the extension in every single database that needs it. Creating an extension is like enabling the usage of the extension within a specific database.

In order to demonstrate the usage of an extension, we will install *orafce*, the Oracle compatibility functions extension. Describing the whole extension is not the aim of this section, so let's just say that this extension provides a set of functions, data types, and other stuff that makes PostgreSQL look like an Oracle database so that migrating an Oracle-based application becomes easier.

The following subsections describe every single step required to get the extension up and running.

Installing the extension via *pgxnclient*

Usually, the first step in installing an extension is getting details about it – that means searching for an extension. In this particular case, we already know what extension we are looking for, but let's search for it via *pgxn*:



Note, *orafce* versions change regularly. 4.5.0 was up to date at the time of writing, but you may see a newer version number.

```
$ pgxn search --ext orafce
orafce 4.5.0
Oracle's compatibility functions and packages
```

The search command explores the ecosystem to find every extension related to our search criteria – in this particular case, the extension name (`--ext`). Thanks to `pgxn`, we now know that we need to install `orafce` version 4.2.1, the latest stable version available at the time of writing.

Once you have decided which extension you need, you can run the `install` command of `pgxn` to let the installation proceed. The installation workflow includes downloading, compiling (if needed) the source tree, packaging it, and placing it in the shared directory of the PostgreSQL cluster.

You can inspect the ongoing process in very rich detail thanks to the `--verbose` option, and if you are using `pgxn` with a different user from the one that runs the cluster, you can use the `--sudo` option to inform `pgxn` to switch to a privileged user when needed:

```
$ pgxn install orafce --verbose --sudo
```



The Docker image used for this chapter has the operating system user allowed to use `sudo` without needing to enter a password. This is *not* a good choice in a production environment but has been used to simplify the experimentation of the chapter examples.

Installing the extension manually

`pgxnclient` is a good tool for automating the installation of extensions, but this does not mean you don't have other choices to improve your PostgreSQL features. Another way to install extensions is by manually downloading them and doing all the steps required to make the cluster aware of the new facilities.

The starting point is the PGXN website, available at <https://pgxn.org>. The site allows you to search for a specific extension by name or by keywords. Once you browse the PGXN site, you have a textbox where you can insert the keyword for the search, and since we already know the extension name, we can choose **Extensions** from the pull-down menu.

The web interface is shown in the following screenshot:



Figure 12.1: The main page of the PGXN website

The result of our search will be displayed, as shown in the following screenshot, so we can enter the extensions page with all the information and the documentation for the installation process:

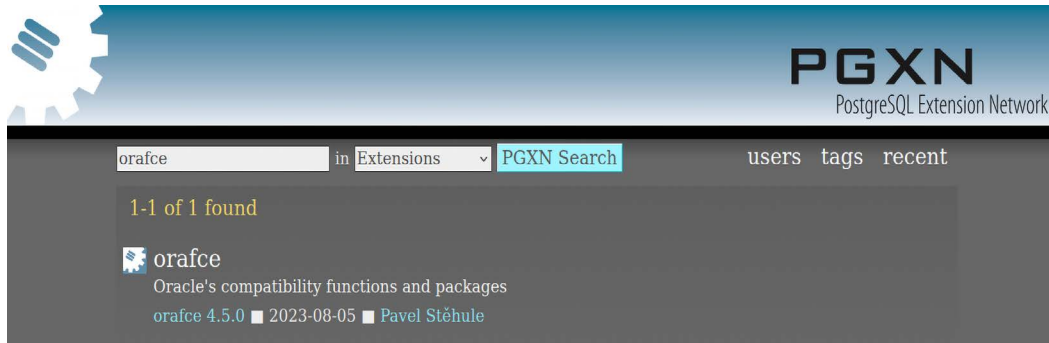


Figure 12.2: PGXN search results page

Once we have found the extension we are looking for, we can download it by clicking on the download icon on the page, like the one shown at the top right in the following screenshot. The result is that we will download a compressed zip file with all the stuff related to the extension:



Figure 12.3: The extension download page

In order to proceed further, you first have to decompress the archive you downloaded:

```
$ unzip orafce-4.5.0.zip
```

Now you can enter the directory created for this extension and compile it (there is the need for a compiler and all the source build tools installed on the system):

```
$ cd orafce-4.5.0
$ make
```



You will need the PGXS Makefiles to compile an extension from scratch. Usually, such Makefiles are installed with the development tools for PostgreSQL. For example, on Red Hat-based Linux distributions, you have to install the `postgresql16-dev` package. In the Docker image of this chapter, all the tools you need to compile and install extensions are already prepared.

If the compilation is successful, the extension is created and ready to be added to the PostgreSQL shared library directory. In order to move the extension files into the cluster, you need access to the database directories, for example, with `sudo`:

```
$ sudo make install
```

From here, you can proceed with the `CREATE EXTENSION` statement in every database that requires the extension.

Using the installed extension

Once the extension has been installed – that means *deployed* to the PostgreSQL cluster either manually or via `pgxn` – you can create the extension in every single database you need it for.

The `orafce` extension must be created by a superuser, so you need to connect to the database as an administrator in order to execute the `CREATE EXTENSION` statement:

```
$ psql -U postgres forumdb
psql (16.0)
Type "help" for help.

forumdb=# CREATE EXTENSION orafce;
CREATE EXTENSION
```

If you now inspect the extensions installed in the database, you will see the freshly created `orafce` at version 4.5.0 – the same as we found when searching the extension with `pgxn` or on the website:

```
forumdb=# \x \dx
Expanded display is on.
List of installed extensions
-[ RECORD 1 ]-----
-----
Name          | orafce
Version       | 4.5
Schema        | public
Description    | Functions and operators that emulate a subset of functions
and packages from the Oracle RDBMS
-[ RECORD 2 ]-----
-----
Name          | pg_stat_statements
Version       | 1.10
Schema        | public
```

```

Description | track planning and execution statistics of all SQL
statements executed
-[ RECORD 3 ]-----
-----
Name       | plpgsql
Version    | 1.0
Schema     | pg_catalog
Description | PL/pgSQL procedural language

```

Once the extension has been installed in the database, every user can use it. As a simple test, you can query the DUAL table that Oracle has and that orafce created for your legacy queries to continue to run:

```

$ psql -U luca forumdb
psql (16.0)
Type "help" for help.

forumdb=> SELECT * FROM oracle.dual;
dummy
-----
X
(1 row)

```

Please note that the tables created by the extension are located in the `oracle` namespace. It is easy to avoid having to type `oracle` before any object name by instead using `search_path`:

```

forumdb=> SET search_path TO "$user", public, oracle;
SET
forumdb=> SELECT * FROM dual;
dummy
-----
X
(1 row)

```

Removing an installed extension

It could happen that you don't need an extension anymore, and therefore you want to remove it from your cluster. Removing unused extensions is a good habit because it keeps the cluster clean and not dependent on objects you really do not need.